

NivXRay · IUE / Workspace / Input Architecture 360° READ-ONLY AUDIT

Type: READ-ONLY architecture discovery. No code, UI, IUE, routing, MITRE, verdict, adapters, or environment modified. **Date:** 2026-02-15 · Session-20 (owner-authorised · post-Task-2). **Method:** Every claim is grounded in a file path + line number + function/class. Anything not verifiable from code is tagged NOT VERIFIED FROM CODE.

§0 · FINAL OWNER QUESTION — ONE-SENTENCE ANSWER

"Is the existing IUE truly the single authoritative Input Understanding → Decision → Routing layer of NivXRay?"

No. At least **eight** independent classifier / understanding / routing modules are currently live in the codebase, and the **primary Workspace Investigate path (/api/decode/smart) bypasses the IUE entirely** — it enters `deterministic_best_decode → smart_decoder + magic_decode` without any IUE call, and the atomic-IOC guard in `v2/investigation/pipeline.py::_atomic_ioc_kind` makes routing decisions on its own before any classifier runs.

Components currently sharing IUE responsibility (evidence in §2 · §3 · §4):

#	File	LoC	Role assumed
1	<code>services/die/input_understanding.py</code>	761	"Official" IUE v2.0 (per header comment). Endpoint POST <code>/api/die/understand</code> .
2	<code>nivxforge/investigation/input_understanding.py</code>	291	Second IUE with its own <code>understand()</code> function. Called by <code>routers/ops.py::decode_smart</code> and <code>routers/auto_investigate.py</code> .
3	<code>services/ida/input_classifier.py</code>	223	IDA classifier (<code>classify_artifact_input</code>). Runs inside <code>/api/die/investigation-results</code> .
4	<code>services/ida/url_intent.py</code>	373	Separate URL-intent classifier with its own vendor catalogue.
5	<code>services/uil/classifier.py</code>	302	UIL classifier with <code>InputKind</code> enum (21 kinds). Endpoint POST <code>/api/uil/classify</code> .
6	<code>v2/investigation/pipeline.py::_atomic_ioc_kind</code>	(th)	Atomic-IOC fast-path decision — routes URL/IP/hash/filename to a short-circuit BEFORE any classifier is consulted.
7	<code>canonical/iue/plan_builder.py</code>	195	Third IUE (canonical plan builder).

8	canonical/ieue/composer.py	285	Fourth IUE (canonical composer).
---	----------------------------	-----	----------------------------------

Combined ~2 727 LoC of classification / routing / decision logic living in parallel. The IUE at (1) is quoted throughout the codebase as "the classifier of record", but the observable runtime behaviour proves (6) fires first for atomic IOCs, (5) fires on its own endpoint, and (2) is what actually runs from the primary Workspace decode path.

§1 · Executive summary

1. **The IUE at `services/die/input_understanding.py` is a real, complete classifier** — 21 first-class input types, 4-layer plan generator, deterministic (no LLM, no network), 98 % confidence on the SystemWeakness URL.
2. It **DOES classify correctly**: `url_only`, confidence 0.98. That part of the machinery is not broken.
3. **The failure is not in classification — it is in the plan the IUE emits for `url_only`**. The plan lists two engines (IOC Enrichment, Report Generator) and explicitly skips ten others (Decoder, DIE, DKP, Chain, Attack Story, Investigation Confidence, Artifact Intelligence, ...). **URL Acquisition is not in the selection**. The IUE therefore never routes `url_only` through the acquisition cascade.
4. In parallel, `POST /api/die/investigation-results` reaches a **second gate** in `services/die/investigation_results.py:322` that requires `BOTH ida_class ∈ {threat_report_url, code_snippet_url, repository_url, file_resource_url}` AND `url_intent.acquirable=True`. This gate is a **default-deny** for any host not in the 43-entry `_VENDORS` catalogue in `services/ida/url_intent.py` — Medium, systemweakness.com, infosecwriteups, etc., all get rejected.
5. **The primary Workspace submit at `/api/decode/smart` does NOT invoke the IUE at all**. It routes through `_atomic_ioc_kind` (short-circuit for atomic IOCs) → `deterministic_best_decode` → `smart_decoder + magic_decode` → `evidence_extractor.build_verdict_card`. IUE-authored `input_understanding` is only attached to the CIO metadata **after** decoding, as an advisory diagnostic (`routers/ops.py:2495`, wrapped in try/except with a "safe" fallback path if IUE fails).
6. **Input Type · Source · Content are collapsed into ONE classification** in every classifier. There is no dimension in the IUE decision object that says "URL is the type, SystemWeakness is the source, HTML + inline base64 + screenshot images is the content". The 21-type taxonomy at `input_understanding.py:37-59` is a flat enum.
7. **The IUE cannot express "URL → article → text + image → OCR → combined evidence."** The URL adapter and image adapter (with Tesseract OCR) exist under `services/adapters/*` but are un-imported from any production path. The IUE plan for `url_only` has no branch for images or OCR.
8. **UI-DEF-02 authoritative MITRE convergence is real** but only reachable from `/api/analyze` and `/api/die/analyze`. The primary Workspace flow still uses `operations.mitre_map()` regex inside `evidence_extractor.build_verdict_card`. **Two MITRE surfaces continue to coexist for the same input**.
9. **Recursive artifact discovery is implemented in `services/die/recursive_decode.py`** but its `_B64_PATTERNS` recognises PowerShell / .NET / bash base64 — NOT Python base64. `b64decode(...)` — and the discovered child artifacts do NOT re-enter through the IUE. They are peeled and re-analysed within DIE only.
10. **A single input can produce four different verdicts / MITRE lists** depending on which endpoint the Workspace calls in parallel: `/api/decode/smart` (regex MITRE), `/api/die/analyze` (DIE catalogue), `/api/analyze` (authoritative surface with `regex_extra` diagnostic), `/api/die/investigation-results` (canonical narrative augmentation). The Workspace merges them client-side.

§2 · The existing IUE — implementation audit

2.1 Files

File	LoC	Purpose
services/die/input_understanding.py	761	Primary IUE v2.0 · classifier + plan generator + summariser + trace executor
services/die/input_health.py	(unread)	Stage-0 pre-IUE health check (called before classify)
services/die/canonical.py	(unread)	Canonical envelope, materialises IUE plan into PlanStep dicts
nivxforge/investigation/input_understanding.py	291	SECOND IUE — separate module, separate signature
canonical/iue/plan_builder.py	195	THIRD IUE — canonical plan builder
canonical/iue/composer.py	285	FOURTH IUE — canonical composer
canonical/iue/adapters/text_structure.py	(unread)	Bridges IUE(die) into canonical/iue at line 10

2.2 Public surface

- **Class:** InputUnderstanding (dataclass at input_understanding.py:122).
- **Function:** understand(text, *, execute=True) at input_understanding.py:705.
- **Sub-functions:**
 - classify(text) at line 213 → returns (input_type, hero, confidence, reasoning[]).
 - _summarize_contents(text, input_type) at line 350.
 - _plan_decode_layers(text, input_type) at line 376.
 - _next_engine(input_type) at line 430 — the routing table.
 - _build_plan(input_type, decode_required, decode_layers) at line 480.
 - _execute_plan(text, plan, input_type, ...) at line 518 — deterministic trace runner.
 - _engines_selected / _engines_skipped / _pipeline_flow at 642 / 679 / 684.

2.3 Endpoints exposing the IUE

- POST /api/die/understand → routers/die.py:62 → die_understand(body) → services.die.understand_input(body.input, execute=body.execute).
- POST /api/understand → routers/ops.py:2760 → calls the **OTHER** IUE (nivxforge.investigation.input_understanding.understand). Same endpoint name, different module.

2.4 Capability matrix (as implemented)

#	Capability	Status	Location	Notes
1	Input identification (any-bytes → type)	■ IMPLEMENTED	classify() @ 213	21 hard-coded types
2	Input classification (choose one)	■ IMPLEMENTED	same	priority-ordered rules

3	Format detection (PE, PDF, RTF, gzip, registry)	■ IMPLEMENTED	221-244	magic-byte checks
4	Source/vendor detection (Talos, Mandiant, ...)	■ BYPASS	delegated to <code>services/ida/url_intent.py</code> — a separate table not owned by IUE	Two <code>vendor</code> catalogues can exist (<code>url_inte</code> + analysis — IUE consults NEITHER
5	Content detection (text + image + encoded)	■ NOT IMPLEMENTED	—	flat taxonomy, no compound content model
6	Encoding detection (base64, hex, gzip)	■ PARTIAL	<code>_looks_like_base64</code> , <code>_hex_ratio</code>	recursive layer detection only inside <code>_plan_dec</code>
7	Artifact detection (child artifacts)	■ NOT IMPLEMENTED IN IUE	recursive discovery lives in <code>services/die/recursive_decode.py</code> and doesn't call IUE	
8	Language detection (PS / Python / bash / JS / VBS)	■ PARTIAL	277-317	PowerShell + interpreter-only; no Python/bas AST detection at IUE layer
9	Intent detection (what to do)	■ PARTIAL	<code>_next_engine()</code> @ 430	1-line reason string per type; no acquisition / <code>ocr_requ</code> flags

10	Adapter/analyzer selection	■ PARTIAL	<code>_next_engine()</code>	hard-coded map; no adapter registry consulted
11	Processing-plan generation	■ IMPLEMENTED	<code>_build_plan()</code> @ 480	4-stage plan (classify → understand → intent → story → report)
12	Recursive artifact routing	■ NOT IMPLEMENTED IN IUE	recursive is DIE-internal only	
13	Confidence/provenance generation	■ PARTIAL	<code>ConfidenceMatrix</code> @ 111	confidence scalar; no per-decision provenance chain
14	Failure/fallback decisions	■ PARTIAL	<code>overall_status</code> , <code>execution_trace</code>	no policy for "acquisition failed" → try alternative"

§3 · The IUE decision contract (as it exists today)

InputUnderstanding dataclass, `input_understanding.py:122-170` (fields verified).

Field	Present?	Producer	Consumer	Runtime note
<code>input_type</code>	■	<code>classify()</code>	Workspace UI, <code>investigation_results.py</code> , <code>canonical.py</code>	21-value enum
<code>hero</code>	■	<code>classify()</code>	UI banner	display-only
<code>confidence</code>	■	<code>classify()</code>	UI	scalar 0.0–1.0
<code>reasoning[]</code>	■	<code>classify()</code>	UI + trace	prose bullets

summary: ContentSummary	■	_summarize_contents()	UI	length, entropy, ascii_ratio
decode_required: bool	■	_plan_decode_layers()	plan builder	
decode_next: str	■	same	UI	
decode_layers: List[DecodeLayerPlan]	■	same	UI + executor	
plan: List[PlanStep]	■	_build_plan()	UI + executor	id, description, engine, status, reason
execution_trace: List[str]	■	_execute_plan()	UI	ordered log lines
overall_status: str	■	_execute_plan()	UI	ok/parti
engines_selected: List[str]	■	_engines_selected()	UI	
engines_skipped: List[str]	■	_engines_skipped()	UI	
pipeline_flow: List[str]	■	_pipeline_flow()	UI	
source (article source / vendor)	■ MISSING	—	—	no field
content (text? image? mixed?)	■ MISSING	—	—	no field
artifacts_expected (list of child types)	■ MISSING	—	—	no field
acquisition_required	■ MISSING	—	—	no field
ocr_required	■ MISSING	—	—	no field
adapter (chosen adapter id)	■ MISSING	—	—	no adapter registry
analyzer (chosen analyzer id)	■ MISSING	—	_next_engine() returns a prose label, not an id	
provenance (decision chain)	■■ IMPLIED	via reasoning[] only	—	prose, not structured

Conclusion: the IUE contract has WHAT / HOW-TO-PLAN, but no separation of TYPE · SOURCE · CONTENT · ARTIFACT · ADAPTER · ACQUISITION-INTENT · OCR-INTENT. Everything is collapsed into `input_type` + a hard-coded next-engine string.

§4 · Workspace entry-point audit (14 paths)

For every path: does IUE run? If not, mark ■ **IUE BYPASS**.

A · Plain text

```
Workspace → POST /api/decode/smart → _atomic_ioc_kind (not atomic) → deterministic_best_decode  
→ build_verdict_card → returned ■ IUE BYPASS (IUE is attached ADVISORILY to CIO metadata after  
decode at ops.py:2495) Parallel: /api/die/understand → IUE ■ (advisory panel) Parallel:  
/api/die/analyze → DIE (no IUE call inside)
```

B · Command line / CMD

Same as A. ■ IUE BYPASS on the primary submit path.

C · PowerShell (naked / -EncodedCommand)

Same as A. IUE classify() correctly identifies powershell_encoded / powershell_naked when invoked, but the primary path bypasses it.

D · Python

Same as A. **Python is not even a first-class IUE type** (the 21-value enum has no python_script — it falls under command_chain or plain_text).

E · Encoded command chain

Same as A. recursive_decode runs inside DIE without IUE participation.

F · URL (bare)

```
Workspace → POST /api/decode/smart → _atomic_ioc_kind(URL) → returns 'url' → SHORT-CIRCUIT:  
engine="-", output=<URL echo>, mitre=[], iocs={} ■ IUE NEVER RUNS on the primary path. Parallel  
→ POST /api/die/understand → IUE ■ → input_type=url_only, confidence=0.98 → next_engine="IOC  
enrichment (URL)" (does not include acquisition) → engines_selected=["IOC Enrichment", "Report  
Generator"] → engines_skipped includes Decoder, DIE, DKP, Chain, Attack Story, Investigation  
Confidence, Artifact Intelligence. Parallel → POST /api/die/investigation-results →  
services/die/investigation_results.py:307 ida_verdict = _ida_classify(src) ← second classifier →  
services/ida/url_intent.py::classify_url_intent _VENDORS table lookup ← third classifier (host  
suffix) systemweakness.com NOT LISTED → intent="atomic_ioc", acquirable=False → gate at  
investigation_results.py:322 fails → acquire NOT called
```

G · File upload

```
Workspace → POST /api/upload → services/files/store::FileStore.put → GridFS →  
services/files/input_router::route_for(magic, mime, name) content-magic routing → returns  
"pe"|"pdf"|"office"|"archive"|"email"|"csv"|"image"|"text" ■ IUE BYPASS — this is a FOURTH  
routing decision, wholly separate from IUE → bytes returned to router which treats them as text  
OR ZIP-extracted
```

H · Sysmon XML paste (BehavioralTimeline)

Workspace → POST /api/behavioral/sysmon → services/behavioral/sysmon_adapter::normalize_sysmon_xml ■ IUE BYPASS – dedicated behavioral lane

I · Sysmon EVTX drop

Workspace → POST /api/behavioral/sysmon/evtx → services/behavioral/evtx_reader → normalizer ■ IUE BYPASS

J · Generic Windows event log (non-Sysmon)

IUE classify() recognises windows_event_log at 252-255 (requires EventID + Provider(Name|Guid) markers). No downstream analyzer wired. UIL classifier (services/uil/classifier.py) has InputKind.EVTX but neither drives a working analyzer. **Full pipe: NOT SUPPORTED.**

K · Image

Workspace → /api/upload → input_router → "image" → no analyzer wired ■ IUE BYPASS – plus dead-end (ImageAdapter exists but never imported)

L · PDF

Workspace → /api/upload → input_router → "pdf" → text extraction only (pdfplumber under the hood) → parallel /decode/smart on extracted text ■ IUE BYPASS on upload; IUE MAY run on the extracted text path

M · DOCX / PPTX / XLSX

Workspace → /api/upload → input_router → "office" → safe_iter_zip_members → text extraction from word/document.xml etc. → text goes through /decode/smart + /die/investigation-results → canonical narrative bridge fires prose-MITRE ■ IUE BYPASS at upload boundary

N · PE binary

Workspace → /api/upload → input_router → "pe" → services/pe_analyzer.py → produces PE header/section summary → NOT WIRED to risk_score or verdict_card ■ IUE BYPASS + orphan analyzer

O · Recursively discovered child artifact (from decoded base64)

DIE analyze → services/die/recursive_decode._B64_PATTERNS (PowerShell -Enc / .NET / bash) → peel base64 → produce DecodedLayer[] → merge_evidence: add T1140 + peel evidence ■ Recursively discovered artifacts do NOT re-enter IUE. ■ The peeled child is analysed by the same DIE call, not routed. ■ Python base64.b64decode is NOT recognised by _B64_PATTERNS.

§5 · Special case — SystemWeakness URL (code-grounded trace)

Input:

<https://systemweakness.com/threat-analysis-report-suspicious-powershell-script-execution-soc153-756d7ad95222>

Observed:

1. Workspace submits URL. 2. POST /api/decode/smart hits routers/ops.py::decode_smart (line ~382). 3. Line 815-816: `_atomic_ioc_kind(body.input)` → returns 'url'. 4. Short-circuit returns: {output: <URL echoed>, engine: '-', confidence:100, mitre:[], iocs:{}, verdict_card: absent}. 5. Saved case ('Examine' · id 2d201527-...) reflects this exactly. Parallel – POST /api/die/understand: 6. routers/die.py:62 → die_understand. 7. services.die.understand_input(url, execute=True) → services/die/input_understanding.py:705. 8. classify() at line 305 matches `r"\s*https?://\S+\s*" → ("url_only", "URL", 0.98, ["Input is a bare URL."])`. 9. `_next_engine("url_only")` at line 468-469 → ("IOC enrichment (URL)", "Single URL – enrichment only."). 10. `_engines_selected("url_only")` → ["IOC Enrichment", "Report Generator"]. 11. `_engines_skipped` includes 10 engines, notably Decoder, DIE, DKP, Chain, Attack Story, Investigation Confidence, Artifact Intelligence. 12. Plan emitted: classify (iue) → understand (preprocessor) → intent (die) → story (attack_story) → report (report). 13. Plan does NOT contain any "URL Acquisition" step. Parallel – POST /api/die/investigation-results: 14. services/die/investigation_results.py:307 → `ida_verdict = _ida_classify(url)`. 15. services/ida/input_classifier.py:130 falls through to "atomic_ioc_url" because services/ida/url_intent.py::`classify_url_intent(url)` can't find "systemweakness.com" in `_VENDORS` at lines 53-96. 16. investigation_results.py:322 requires `ida_class` in {threat_report_url, code_snippet_url, repository_url, file_resource_url} AND `url_intent.acquirable=True`. 17. Both fail → `_ida_acquire(url)` is NEVER called. 18. Response body contains no article text, no extracted commands, no extracted IOCs.

WHY does the system fail to reach "this is a security research / threat-report URL containing potentially analyzable content"?

The root cause is **NOT a single component**. It is the compound effect of:

1. **IUE plan-table gap** — services/die/input_understanding.py:468 maps `url_only` → "IOC enrichment (URL)". There is no branch that routes any URL through the acquisition pipeline. IUE decision layer, not classification.
2. **IDA URL-intent catalogue gap** — services/ida/url_intent.py:53-96 `_VENDORS` list has 43 hosts, missing medium.com, systemweakness.com, infosecwriteups.com, posts.specterops.io, etc. Vendor-catalogue policy layer.
3. **Gate composition** — services/die/investigation_results.py:322 requires BOTH `ida_class` in the acquirable set AND `url_intent.acquirable=True`. Default-deny composition.
4. **Workspace primary route (/api/decode/smart) bypasses IUE entirely** — the atomic-IOC guard at v2/investigation/pipeline.py::`_atomic_ioc_kind` short-circuits URL inputs to engine="-" BEFORE the IUE (or IDA) has any say. Routing/architecture layer.
5. **Even if all above were fixed, acquire_url returns HTTP 403 from Medium** — services/ida/acquisition.py sends the default requests User-Agent and Medium/Cloudflare block it. Transport layer.
6. **Playwright fallback silently returns "** — Playwright chromium binaries are not installed in the preview pod, and `_playwright_probe` at acquisition.py:427 silently reports unavailable. Environment layer.
7. **No image OCR** — the PowerShell command in the article is a screenshot; services/adapters/image_adapter.py (Tesseract OCR) is un-imported from any production path. Missing capability.

Six independent causes across five architectural layers. No single one is "the" cause. The IUE-plan-table gap (#1) is the most upstream and would be the correct locus of a fix, because it is the only place that would cause even a KNOWN-vendor URL to skip acquisition today.

§6 · Input Type ≠ Source ≠ Content (dimensional collapse audit)

Current architecture collapses these dimensions in every classifier.

Concept	IUE (die)	IUE (nivxforge)	IDA	UIL	Truth
Input Type (URL / File / Text / EVTX)	input_type (21 values)	type (~15 values)	ida_class (~10)	InputKind (21 values)	Independent axis
Source (SystemWeakness / Talos / Sysmon / SentinelOne)	■ not modeled	■	via _VENDORS in url_intent — 43 hosts, doesn't feed IUE	■	Independent axis
Content (HTML / text / image / OOXML / encoded / mixed)	■ collapsed into input_type	■	■	■	Independent axis
Artifact (child artifacts discovered)	■ not modeled at IUE layer	■	artifacts[] on classifier response — flat list	■	Independent axis
Adapter (URL adapter / PDF adapter / ...)	■ no adapter registry consulted	■	■	■	Independent axis
Acquisition intent (fetch / not fetch)	■ implicit in _next_engine label only	■	url_intent.acquirable: bool	■	Independent axis
OCR intent (needs OCR / doesn't)	■ not modeled	■	■	■	Independent axis

Example: EVTX file containing Sysmon Event 1 + Event 3 — the current model represents this as input_type=sysmon_log (die IUE) OR InputKind.EVTX (UIL) OR route_for()="text" (input_router). It does **not** simultaneously represent {input=EVTX_file, source=Sysmon, content=[E1,E3], artifact=EVTX_container}.

Example: URL to SystemWeakness article with text + image screenshots — the current model can only say input_type=url_only. It cannot express that the CONTENT will be {html_text, encoded_commandlines, image_with_command}.

Result: **downstream engines cannot make content-aware decisions** because the dimensions never arrive intact.

§7 · URL content decision audit

Question: Can the current architecture express the following?

URL → acquisition → HTML → plaintext → command extraction → DIE → encoded commandline → decode → DIE → images → OCR → merged evidence → linked artifacts → Artifact Router → per-analyzer

Findings:

Capability	Present in code?	Wired into IUE?	Wired into Workspace?
------------	------------------	-----------------	-----------------------

URL acquisition (Trafilatura → readability → BS4 → Playwright)	■ <code>services/ida/acquisition.py.acquire_url</code>	■ IUE plan for <code>url_only</code> does not include <code>url</code>	■■ only fires from <code>/die/investigation-results</code> behind the <code>_ACQUIRABLE_CLASSES</code> gate
Body-text command extraction	■ <code>services/ida/report_extractors.py::extract_all</code>	■	■■ same gate
Body-text IOC / MITRE / actor / malware / CVE extraction	■ same	■	■■ same gate
Encoded command-line detection within acquired body	■■ <code>services/die/recursive_decode._B64_PATTERNS</code> — PS / .NET / bash only, no Python	■	inside DIE, not IUE
Image identification within acquired body	■ NOT PRESENT	■	■
Image download	■ NOT PRESENT	■	■
OCR (Tesseract)	■ <code>services/adapters/image_adapter.py::ImageAdapter</code> (~440 LoC)	■	■ un-imported from production
Merge OCR-derived + body-text-derived evidence	■ NOT PRESENT	■	■
Provenance for OCR vs body-text sources	■ NOT PRESENT	■	■
Linked artifact download	■■ partial via <code>acquired.outbound_links</code> field (never re-entered pipeline)	■	■
Fixed-point termination for artifact recursion	■ NOT PRESENT	■	■

Missing boundary: there is no `ContentDecomposer` component between `acquire_url` and the analyzers. Everything downstream of `acquire_url` receives `article_text: str + structured_blocks: List[str]` — a flat text projection. Images, embedded objects, and linked artifacts are dropped at that boundary.

§8 · Text + Image provenance audit

Canonical evidence today (per `services/behavioral/sysmon_adapter.py` and `services/uaie/evidence.py`) has:

```
{source, event_or_rule, field, observed_value, evidence_ref, confidence, provenance, raw_refs[]}
```

`source` can currently take values like `"sysmon.event1"`, `"sysmon.event3"`, `"csv_edr.symantec"`, `"die.analyzer_catalogue"`, `"ai"`. There is **no** `source="image_ocr"` or `extraction_method="ocr"` **value** anywhere in the code.

`services/adapters/image_adapter.py:305-325` internally emits `image_ocr` tags and `ocr_confidence` scalars, but because the adapter is un-imported from production, those never reach canonical evidence.

Text-vs-OCR provenance today:

Provenance field	Present today?	Notes
source = "article_text"	■	acquisition never populates a canonical evidence record; the extracted body goes directly into IDA report-extractors, which produce artifact records without a source="article_text" tag
source = "image_ocr"	■	ImageAdapter is dead-code
extraction_method field	■	not modeled
image_ref / image_hash	■	not modeled
ocr_confidence	■■ modeled in ImageAdapter (dead-code)	never reaches canonical evidence
Merge / dedup between text-derived and OCR-derived	■	no merger — the two lanes don't converge because OCR lane is dead

Risk: if OCR were wired in tomorrow without provenance fields, OCR text would be **indistinguishable from authoritative body text**. That is an evidence-integrity risk that the current schema cannot express.

\$9 · Adapter / analyzer routing map

Each row shows one input → the actual runtime path.

Input	Router / decision	Adapter	Normalizer	Analyzer	Canonical evidence
PowerShell cmd	deterministic_best_decode + IUE(shadow)	(no adapter)	(no norm)	services/die/powerShell.py	DIE envelope
CMD	same	—	—	services/die/cmd.py	DIE envelope
Python	same	—	—	services/die/python (partial)	DIE envelope
JS / VBS / bash	same	—	—	corresponding services/die/*_ast.py	DIE envelope
Base64 blob	deterministic_best_decode → recursive_decode	—	—	DIE post-peel	DIE envelope + T1140
URL (recognised vendor)	_ida_classify → _ida_acquire → _ida_extract → _ida_investigate_all (via /die/investigation-results only)	(no adapter registry)	—	DIE on extracted commands	canonical narrative augmented

URL (unrecognised host)	<code>_atomic_ioc_kind</code> — → <code>engine='- '</code> ; parallel <code>_ida_classify</code> → <code>atomic_ioc_url</code>	—	—	none	empty
Sysmon XML	<code>dedicated</code> <code>/api/behavioral/sysmon</code> route	—	<code>services/behavior/authoritative_techiniques</code> (DIE on E1 CommandLine)	same as behavioral evidence	
Sysmon EVTX	<code>dedicated</code> <code>/api/behavioral/sysmon/evtx</code>	—	<code>services/behavior/safe/evtx_reader.py</code> → normalizer	same + transport meta	
PE upload	<code>services/files/input_router.route</code>	—	<code>services/pe_analyser.py</code> (dead)	—	
DOCX upload	same	—	zip-based text extraction	canonical narrative bridge	
PDF upload	same	—	pdfplumber (in-line)	text pipeline	
Image upload	same	—	<code>services/adapters/image_adapter.py</code> (dead)	—	
Archive (ZIP)	same	—	<code>safe_iter_zip_member</code> pipeline	via /decode/smart	
Legacy RC2 orchestrator	<code>rc22_adapter.try_orchestrator_first</code> inside <code>deterministic_best_decode</code>	—	—	RC2.2 pipeline	
Legacy MITRE mapper	inside <code>evidence_extractor.build_verdict_card</code>	—	—	<code>operations.mitre_mitre[]</code> on regex /decode/smart	

Routing is **hard-coded and endpoint-specific**. There is no single adapter registry that every entry point consults. The `services/adapters/*` package (URLAdapter, ImageAdapter, PDFAdapter, DOCXAdapter, EMLAdapter, ZIPAdapter, TextAdapter) is a **complete SHADOW registry** — 7 well-structured adapters, all inheriting from EvidenceAdapter, all **un-imported from production code**.

Competing implementations (same job, multiple code sites):

Job	Competing implementations
Input classification	8 (see §0)
URL classification	<code>services/die/input_understanding.py:305</code> (regex) + <code>services/ida/input_classifier.py:86</code> + <code>services/ida/url_intent.py:187</code> + <code>nivxforge/investigation/input_understanding.py</code> + <code>_atomic_ioc_kind</code> + <code>services/uil/classifier.py</code> (InputKind.URL) — 6 sites
MITRE mapping	11 (see ADR-0012 §6)
Verdict scoring	3 (<code>risk_score</code> + <code>build_verdict_card</code> + shadow v3)
Attack Chain rendering	2 (AttackGraph legacy + TrajectoryDiagram modern)

Narrative	2 (services/die/narrative.py legacy + canonical_narrative_enrichment)
-----------	---

§10 · Artifact-first / recursive discovery

Live paths:

```
DIE analyze (services/die/api.py::analyze) ■ ■■■■ _analyze_single(src, language) ■ →
per-language AST (powershell_ast / cmd_ast / ...) ■ → produces techniques[], iocs[], lolbins[],
commands[] ■ ■■■■ _apply_recursive_decode(env, src) →
services/die/recursive_decode.py::extract_decoded_layers (recognises PowerShell -Enc / .NET
FromBase64String / bash base64 -d ONLY) → for each layer, re-runs analyze on decoded content →
merge_evidence back into envelope
```

IUE participation in recursion? ■ None. Recursive decode is a DIE-internal loop; IUE is not consulted for the child layer.

Artifact Router participation in recursion? ■ None. There is no artifact router in the running system; the concept exists only in ADR-0008 aspirational text (`_ADAPTERS` under `services/adapters/*` is a registry stub, not a router).

Can child artifacts re-enter through IUE? ■ No. The child bytes go straight to the same DIE analyzer that peeled them. If the child is a DOCX, PE, PDF, or image, the DIE analyzer has no way to route it to the correct analyzer — it treats it as text.

Fixed-point termination: `recursive_decode.py:158` caps recursion at 5 layers hard-coded. No dedup on peeled content.

§11 · Authoritative investigation convergence

11.1 Where each step happens

Step	Authoritative location	Legacy sibling(s) still live?
Evidence normalisation	services/behavioral/sysmon_adapter.py (Sysmon lane); services/die/api.analyze (DIE lane); IDA report-extractors (URL lane)	■ separate lanes still live
Correlation	services/correlation_engine.py; services/ice/correlate.enrich_clusters_in_place (on /api/cases/{id} read)	—
Signal generation	services/die/api.analyze::_analyze_single + LOLBAS + DKP	■ legacy evidence_extractor.build_verdict_card
MITRE technique resolution	analysis_core.get_authoritative_mitre → DIE catalogue + P0.2 gate	■ operations.mitre_map regex still fires inside verdict_card
Verdict scoring	operations.risk_score (recalibrated Item-1)	■■ shadow Verdict Engine v3
Narrative	services/die/narrative.py + canonical_narrative_enrichment (composed)	—

Report generation	<code>routers/reports.py</code> (determinism-locked)	■ 6 legacy <code>/api/report*</code> routes flagged for sunset per ADR-0009 §5.1
IKG	<code>services/uaie/*</code> + <code>services/attack_fingerprint.py</code> (SHADOW)	—
Workspace projections	<code>WorkspacePage.jsx</code> merge of 5 parallel API responses	■ <code>AttackGraph</code> legacy still imported

11.2 Do ALL Workspace inputs converge on UI-DEF-02 authoritative MITRE?

NO.

- `/api/decode/smart` → `verdict_card.mitre` comes from `operations.mitre_map` (regex).
- `/api/analyze` → converges via `get_authoritative_mitre` + gate.
- `/api/die/analyze` → DIE catalogue directly.
- `/api/die/investigation-results` → DIE catalogue + canonical narrative bridge additions.
- `/api/behavioral/sysmon(*)` → DIE catalogue via Event-1 CommandLine hand-off; E3 alone emits nothing.

The Workspace fires **all five in parallel** and merges client-side. The MITRE list shown to the analyst depends on which panels are visible and in what order. **The saved case's `mitre[]` reflects the `/decode/smart` output only** — that's what PrevMode and Examine both demonstrate.

§12 · Empty-result / failure semantics

Current behaviour when the pipeline fails to produce evidence:

Failure mode	Workspace observation	Distinguishable?
A. No analyzable evidence found	empty panels + verdict card either absent or Suspicious-low	■ collapses with B/C/D/E
B. Acquisition returned HTTP 403	empty panels; <code>acquired.error_code =</code> <code>"http_error"</code> inside investigation-results response, but the Workspace does not surface this to the analyst	■ silent
C. Acquisition succeeded but extraction failed	empty panels	■ silent
D. Input was intentionally treated as atomic IOC	empty panels + engine: <code>" - "</code> in saved case	■ visible only via case save
E. Analyzer was not invoked (IUE plan skipped it)	empty panels	■ silent
F. Playwright unavailable	empty panels (fallback returned <code>"</code>)	■ silent
G. Article contained only images	empty panels (no OCR wired)	■ silent

H. Timeout on TI / OSINT / AI describe	verdict card + ti_lookup_meta.status: timeout chip	■■ chip only, no panel
--	--	------------------------

Six failure modes collapse into one visible outcome — an empty investigation. The analyst cannot distinguish between "the IUE saw the URL and decided it's IOC-only" and "the URL was fetched but Cloudflare blocked the transport" and "the URL was fetched but the vendor catalogue didn't recognise it as a threat report".

This is an architectural UX correctness problem, not a rendering bug. Documented, not fixed.

§13 · Compatibility matrix

Legend: ■ FULL · ■ PARTIAL · ■ PENDING · ■ BYPASS/CONFLICTING · ■ DEAD/DEAD-ISH · ■ UNKNOWN.

Commands / scripts

Input	IUE	Classification	Decision	Adapter	Analyzer	Canonical Evidence	MITRE	Verdict	Workspace
CMD	■ bypassed on primary	■ IUE recognises	via decode_smart	none	cmd_ast.py	via DIE	■ split	■	■
PowerShell	■ bypassed	■	via decode_smart	none	powershell_ast.py	via DIE	■ split	■	■
Python	■ bypassed	■ (no Python-specific IUE type)	via decode_smart	none	python_ast.py (partial)	via DIE (missing T1059.006/T1027.013/T1620)	■ split	■	■
Bash	■ bypassed	■	via decode_smart	none	bash_ast.py	via DIE	■ split	■	■
Shell	■ bypassed	■	via decode_smart	none	shared bash_ast	via DIE	■	■	■
SQL	■ bypassed	■	via decode_smart	none	—	—	■	■	■

Windows

| Sysmon XML | ■ dedicated route | ■ | ■ | sysmon_adapter | E1/E3 normalizer | ■ | ■ (via E1) | ■ not fed to verdict | ■ Timeline | | Sysmon EVT_X | ■ dedicated route | ■ | ■ | evt_x_reader → normalizer | same | ■ | ■ | ■ | ■ | | Windows Event (non-Sysmon) | ■ bypass | ■ (IUE recognises) | ■ dead-end | none | none | — | — | — | ■ | | WMI | ■ | ■ | ■ | ■ | ■ | ■ | — | — | — | — | ■ |

Linux / Unix

| Syslog | ■ | ■ | ■ | ■ | ■ | — | — | — | ■ | | Linux auth | ■ | ■ | ■ | ■ | ■ | — | — | — | ■ | | auditd | ■ | ■ | ■ | ■ | ■ | — | — | — | ■ | | journald | ■ | ■ | ■ | ■ | ■ | — | — | — | ■ |

Network / security

| Firewall | ■ | ■ | ■ | ■ | ■ | — | — | — | ■ | | DNS | ■ (Slice-4 LOCKED) | ■ | ■ | ■ | ■ | — | — | — | ■ | | Proxy | ■ | ■ | ■ | ■ | ■ | — | — | — | ■ | | Web server | ■ | ■ | ■ | ■ | ■ | — | — | — | ■ | | EDR (structured) | ■ (CSV only via csv_edr) | ■ (Symantec SEP only) | ■ | none | csv_edr_analyzer | ■ SEP only | ■ SEP mapped | ■ | ■ SEP only | | XDR / MDR | ■ | ■ | ■ | ■ | ■ | — | — | — | ■ |

Formats

| JSON | ■ bypass | ■ (ingress-gate normalises 7 vendors) | via decode_smart | none | text pipeline | ■ | ■ | ■ | ■ | | NDJSON | ■ | ■ | ■ | ■ | ■ | — | — | — | ■ | | XML | ■ (Sysmon-shaped only) | ■ | via /behavioral | sysmon | ■ | ■ | ■ (E1) | — | ■ | | CSV | ■ bypass | ■ | via decode_smart | none | csv_edr (SEP only) | ■ | ■ | ■ | ■ | | CEF | ■ | ■ | ■ | ■ | ■ | — | — | — | ■ | | LEEF | ■ | ■ | ■ | ■ | ■ | — | — | — | ■ | | STIX 2.1 | ■ | ■ UIL InputKind.STIX exists | ■ | ■ | ■ | — | — | — | ■ |

Artifacts

| PDF | ■ bypass (upload → route_for) | ■ | text-extract only | pdfplumber inline | text pipeline | ■ | ■ | ■ | ■ | | DOCX | ■ bypass | ■ | ZIP text extract | inline | prose narrative bridge | ■ | ■ | ■ | ■ | | XLSX | ■ bypass | ■ | sharedStrings only | inline | text pipeline | ■ | ■ | ■ | ■ | | PPTX | ■ bypass | ■ | slide XML only | inline | text pipeline | ■ | ■ | ■ | ■ | | PE | ■ bypass | ■ (input_router) | orphan | pe_analyzer | none-to-verdict | ■ | — | — | ■ | | ELF | ■ | ■ | ■ | ■ | ■ | — | — | — | ■ | | Image | ■ bypass | ■ (input_router) | dead-end | ImageAdapter ■ | none | — | — | — | ■ | | Archive (ZIP) | ■ bypass | ■ | ZIP iteration | inline | text pipeline | ■ | ■ | ■ | ■ | | Archive (7z/RAR/ISO) | ■ | ■ | ■ | ■ | ■ | — | — | — | ■ | | Email (.eml/.msg) | ■ | ■ UIL InputKind + input_router "email" | ■ | ■ dead | ■ | — | — | — | ■ | | PCAP | ■ | ■ UIL InputKind.PCAP | ■ | ■ | ■ | — | — | — | ■ |

URLs

| Atomic IOC URL | ■ IUE + IDA | ■ both agree | ■ both agree | none | none needed | ■ empty | — | — | ■ (empty case saved as Examine) | | Threat-report URL (known vendor) | ■ IUE plan lacks acquisition | ■ IDA _VENDORS catches | ■ gate passes | none | _ida_extract | ■ | ■ | ■ | ■ | | Threat-report URL (unknown vendor, e.g. SystemWeakness) | ■ BYPASS + default-deny | ■ IDA rejects | ■ gate fails | none | ■ | ■ empty | ■ | ■ | ■ empty case | | Security research (blog / gist / paste) | ■ same problem as above | ■ | ■ | none | ■ | — | — | — | ■ | | Repository (GitHub / GitLab) | ■ IDA has repository_url intent | ■ depends on host | ■ depends on host | none | limited | ■ | ■ | ■ | ■ | | File-resource URL | ■ IDA has file_resource_url intent | ■ depends on host | ■ | none | limited | ■ | ■ | ■ | ■ | | Generic webpage | ■ | ■ falls to atomic_ioc_url | ■ default-deny | none | ■ | — | — | — | ■ | | Inaccessible URL (403/404/timeout) | ■ | ■ fetch fails silent | ■ | none | ■ | — | — | — | ■ | | Unknown URL | ■ | ■ | ■ | none | ■ | — | — | — | ■ |

§14 · Architecture diagrams (Mermaid — CURRENT SYSTEM)

A · Workspace → IUE → analysis (as it actually is)

```
graph TD
    U[Workspace Input] --> Wchoice{Path chosen<br/>by Workspace}
    Wchoice --> Primary|Primary|
    Wchoice --> ParallelEnrichment1[Parallel enrichment]
    Wchoice --> ParallelEnrichment2[Parallel enrichment]
    Wchoice --> ParallelEnrichment3[Parallel enrichment]
    Wchoice --> ParallelEnrichment4[Parallel enrichment]
    Wchoice --> AIoc{atomic_ioc_kind?}
    AIoc --> AIocYes|yes|
    AIoc --> AIocNo|no|
    AIocNo --> DBD[deterministic_best_decode<br/>smart_decode + magic_decode + rc22]
    DBD --> VC[evidence_extractor.build_verdict_card<br/>+ operations.mitre_map REGEX]
    VC --> WSPanel1[Workspace verdict panel<br/>MITRE (legacy)]
    WSPanel1 --> DS_smart[Primary]
    WSPanel1 --> DS_smart2[ops.py:2495]
    WSPanel1 --> IUE1[nivxforge IUE.understand<br/>(SECOND IUE)]
    IUE1 --> CIOMeta[cio.metadata.input_understanding]
    CIOMeta --> DU[DU]
    DU --> IUE2[services.die.IUE.understand<br/>OFFICIAL IUE]
    IUE2 --> Plan[21-type classify + 4-stage plan]
    Plan --> DA[DA]
    DA --> DIE[services.die.api.analyze<br/>authoritative catalogue]
    DIE --> UIDEF02[get_authoritative_mitre<br/>+ P0.2 evidence gate]
    UIDEF02 --> DIR[DIR]
    DIR --> IDACls[services.ida.classify_artifact_input<br/>THIRD classifier]
    IDACls -->
```

```
URLIntent["services.ida.url_intent<br/>_VENDORS host table"] URLIntent --> AcqGate{acquire?}
AcqGate -->|known vendor + acquirable| Acq["services.ida.acquire_url<br/>Trafilatura →
readability → BS4 → Playwright"] AcqGate -->|unknown host| Empty2["no acquisition"] style IUE2
fill:#c8e6c9 style IUE1 fill:#fff3cd style IDACls fill:#fff3cd style URLIntent fill:#fff3cd style
AIoc fill:#ffe0e0 style ShortCircuit fill:#ffcdd2 style Empty2 fill:#ffcdd2 style CIOMeta
fill:#eeeeee
```

B · URL → acquisition → content decision (current)

```
flowchart LR URL[URL Input] --> Cls{Which classifier<br/>runs first?} Cls -->|/decode/smart|
At[_atomic_ioc_kind=url<br/>SHORT-CIRCUIT] Cls -->|/die/understand| I[url_only 0.98<br/>plan: IOC
Enrichment only] Cls -->|/die/investigation-results| IDA[_ida_classify] IDA --> UI[url_intent
lookup<br/>43-host _VENDORS] UI -->|known| ACQ[acquire_url<br/>text extraction only] UI
-->|unknown| Drop[atomic_ioc_url<br/>NO acquisition] ACQ --> Body[article_text +
structured_blocks] Body --> Extr[extract_all<br/>commands / IOCs / MITRE / actors] Body
-->|images dropped| Nowhere[■ no image handling] Extr --> DIEsig[per-command DIE analysis] style
At fill:#ffcdd2 style Drop fill:#ffcdd2 style Nowhere fill:#ffcdd2 style I fill:#fff3cd style UI
fill:#fff3cd
```

C · Log → adapter → canonical evidence (current)

```
flowchart LR L[Log Input] --> R{Log kind?} R -->|Sysmon XML paste| SX["/api/behavioral/sysmon"] R
-->|EVTX drop| EV["/api/behavioral/sysmon/evtx"] R -->|JSON telemetry| JS["/api/decode/smart"] R
-->|CSV EDR| CS["/api/decode/smart"] R -->|Windows Event non-Sysmon| WE["■ dead-end"] R
-->|Syslog / WMI / firewall / DNS| SW["■ unsupported"] SX -->
Norm1["sysmon_adapter.normalize_sysmon_xml"] EV --> ER["evtx_reader → sysmon_adapter"] JS -->
IG["ingress_gate JSON normalisation"] --> Text["text pipeline"] CS -->
CE["csv_edr_analyzer<br/>SEP schema only"] Norm1 --> CanEv["canonical behavioral evidence<br/>+
per-event MITRE"] ER --> CanEv Text --> Text2["decoder + DIE + regex MITRE"] CE --> DIE2["DIE +
canonical narrative"] style WE fill:#eeeeee style SW fill:#eeeeee
```

D · Artifact → Artifact Router → analyzer (current)

```
flowchart LR A[Artifact bytes] --> IR["services/files/input_router.route_for<br/>content-magic
table"] IR --> Fork{route} Fork -->|pe| PE[pe_analyzer<br/>orphan – not fed to verdict] Fork
-->|pdf| PDF[pdfplumber text extract] Fork -->|office| OFF[ZIP text extract] Fork -->|archive|
ARCH[safe_iter_zip_members] Fork -->|image| IMG[■ ImageAdapter exists but un-imported] Fork
-->|email| EM[■ no email adapter] Fork -->|csv/text| TXT[text pipeline] A
-->|services/adapters/*<br/>SHADOW registry – 7 adapters| Shadow[URLAdapter · PDFAdapter ·
DOCXAdapter · EMLAdapter · ZIPAdapter · ImageAdapter · TextAdapter<br/>NEVER IMPORTED FROM
PRODUCTION] style Shadow fill:#eeeeee style IMG fill:#eeeeee style EM fill:#eeeeee
```

E · Recursive artifact discovery (current)

```
flowchart LR S[Source text] --> RD["services/die/recursive_decode<br/>_B64_PATTERNS: PS -Enc /
.NET / bash base64 -d"] RD -->|match| Peel[extract Layer N] Peel --> RA[re-analyze via
services.die.api.analyze] RA --> Merge[merge_evidence + T1140] Merge --> RD RD -->|no Python
match| MissPython[■ Python base64.b64decode NOT recognised] RD -->|IUE NOT consulted| BypassI[■
Child artifacts NEVER re-enter IUE] style MissPython fill:#ffcdd2 style BypassI fill:#ffcdd2
```

F · Evidence → correlation → MITRE → verdict → Workspace

```
flowchart LR E[Evidence records] --> C[correlation_engine + ice.correlate on read] C -->
MITRE{Which MITRE surface?} MITRE -->|from /decode/smart path| Regex["operations.mitre_map
REGEX"] MITRE -->|from /analyze path| Auth["get_authoritative_mitre<br/>UI-DEF-02"] MITRE
```

```
-->|from /die/analyze path| DieCat["DIE catalogue direct"] MITRE -->|from /die/narrate path|
Narr["canonical_narrative_enrichment"] MITRE -->|from /behavioral path| Beh["per_event_mitre from
E1 command line"] Regex --> VCard["verdict_card (legacy)"] Auth --> VAnal["/analyze response"]
DieCat --> VDie["/die/analyze response"] Narr --> VNarr["/die/narrate response"] Beh -->
VBeh["BehavioralTimeline"] VCard --> WSMerge["Workspace client-side merge<br/>4-5 responses
combined"] VAnal --> WSMerge VDie --> WSMerge VNarr --> WSMerge VBeh --> WSMerge style Regex
fill:#fff3cd style VCard fill:#fff3cd
```

§15 · Target architecture (DESIGN ONLY — NOT IMPLEMENTED)

The target concept preserves the existing IUE and does NOT create another intelligence layer. It expands the IUE's decision contract and re-wires every entry point to consult it.

```
flowchart TB
  AI[ANY INPUT] --> IUE["IUE (existing<br/>services/die/input_understanding.py)"]
  IUE --> Dec["EXTENDED DECISION CONTRACT<br/>type + source + content[] + artifacts[] +<br/>adapter_id + acquisition_intent + ocr_intent + provenance"]
  Dec --> UIR["UNIVERSAL INPUT ROUTING<br/>(new: single registry lookup, no new engine)"]
  UIR --> CMD["COMMAND / SCRIPT lane<br/>services/die/api.analyze"]
  UIR --> LOG["LOG lane<br/>sysmon_adapter . evt_x_reader<br/>syslog . wmi . edr . firewall . dns - future"]
  UIR --> ART["ARTIFACT lane<br/>services/adapters/* registry -<br/>URL . PDF . DOCX . EML . ZIP . Image . Text"]
  CMD --> CE["CANONICAL EVIDENCE"]
  LOG --> CE
  ART --> CE
  CE --> COR["CORRELATION"]
  COR --> AM["AUTHORITATIVE MITRE<br/>get_authoritative_mitre + P0.2 gate"]
  AM --> V["VERDICT<br/>operations.risk_score - single source"]
  V --> WS["WORKSPACE - one canonical response"]
```

URL / article content sub-flow (design only):

```
flowchart TB
  U2[URL] --> IUE2[IUE]
  IUE2 --> CU["CONTENT UNDERSTANDING<br/>(new IUE branch - NOT a new engine)"]
  CU --> P[plaintext?]
  CU --> EC[encoded commandlines?]
  CU --> IMG2[images?]
  CU --> LA[linked artifacts?]
  P --> CX[command extraction]
  EC --> DIE3[existing DIE]
  IMG2 --> OCRd[OCR decision +<br/>existing ImageAdapter re-wired]
  LA --> ART2[Artifact Router]
  CX --> PA["PROVENANCE-AWARE EVIDENCE<br/>source=article_text | image_ocr"]
  DIE3 --> PA
  OCRd --> PA
  ART2 --> PA
  PA --> EI[EXISTING INVESTIGATION ENGINE]
```

Non-goals of the target design:

- No new IUE.
- No new analysis engine.
- No new MITRE mapper.
- No new verdict engine.
- Do not remove any existing analyzer.
- Do not modify canonical evidence beyond adding provenance fields.

Minimum contract changes required (for reference — NOT authorised for implementation):

1. Extend `InputUnderstanding` dataclass with `source: str`, `content: List[str]`, `artifacts_expected: List[str]`, `adapter_id: str`, `acquisition_intent: bool`, `ocr_intent: bool`.
2. Rewire `_next_engine("url_only")` to return an acquisition-aware plan.
3. Import `services/adapters/*` from a single production module.
4. Add source value `"article_text"` and `"image_ocr"` to canonical evidence provenance.
5. Merge the two IUE modules (`nivxforge` + `services/die`) into one canonical implementation.

§16 · Most important questions — direct answers

#	Question	Answer
A	Is IUE currently authoritative?	NO. Two IUEs (services/die, nivxforge) + 6 other classifiers coexist. The primary Workspace path invokes IUE only advisoirily.
B	Which Workspace inputs bypass IUE?	Every input to /api/decode/smart (the primary Investigate route), /api/upload, /api/behavioral/sysmon(*), plus the atomic-IOC short-circuit for URL/IP/hash/filename.
C	Which analyzers execute without IUE?	deterministic_best_decode, smart_decoder, magic_decode, rc22_adapter, sysmon_adapter, evtx_reader, pe_analyzer, csv_edr_analyzer, operations.mitre_map, operations.risk_score, evidence_extractor.build_verdict_card, services/die/api.analyze (called directly from /die/analyze), canonical_narrative_enrichment.
D	Which components independently classify inputs?	(i) services/die/input_understanding.py::cla (ii) nivxforge/investigation/input_understand (iii) services/ida/input_classifier.py::classifi (iv) services/ida/url_intent.py::classify_url (v) services/uil/classifier.py::classify (InputKind enum), (vi) v2/investigation/pipeline.py::_atomic_io (vii) services/files/input_router.py::route_fo (viii) canonical/iue/plan_builder.py.
E	Which components independently make routing decisions?	The same 8, plus the hard-coded _next_engine map in IUE(die), plus the _ACQUIRABLE_CLASSES gate at investigation_results.py:322, plus the _atomic_ioc_kind short-circuit at ops.py:815.
F	Which legacy routes compete with IUE?	/api/decode/smart, /api/decode/chain, /api/decode/magic, 6 legacy /api/report* routes (ADR-0009 §5.1), 4 /api/timeline/* duplicates (ADR-0009 §5.2).

G	Where does URL classification happen outside IUE?	(i) <code>_atomic_ioc_kind</code> at <code>v2/investigation/pipeline.py:77</code> , (ii) <code>_ida_classify</code> → <code>classify_url_intent</code> at <code>services/ida/url_intent.py:187</code> , (iii) <code>route_for</code> at <code>services/files/input_router.py:29</code> .
H	Where does artifact classification happen outside IUE?	<code>services/files/input_router.py::route_for</code> (content-magic table) and the un-imported <code>services/adapters/*</code> registry.
I	Where does MITRE mapping happen outside the authoritative surface?	(i) <code>operations.mitre_map()</code> regex (still fires inside <code>verdict_card</code>), (ii) <code>canonical_narrative_enrichment</code> (additive), (iii) <code>csv_edr_analyzer</code> (SEP-specific), (iv) <code>services/technique_detector</code> (legacy, dead-ish), (v) AI describe leg.
J	Where does verdict logic happen outside the canonical engine?	(i) <code>evidence_extractor.build_verdict_card</code> (legacy), (ii) shadow Verdict Engine v3 in <code>services/uaie/*</code> , (iii) AI verdict leg on <code>/api/analyze</code> .
K	Can the same input produce different results depending on Workspace entry path?	YES. The Examine case proves this: <code>/decode/smart</code> → empty; <code>/die/investigation-results</code> → also empty because of the vendor catalogue gap; but on a KNOWN vendor URL these two paths would produce completely different envelopes.
L	Can the same artifact be analysed by competing engines?	YES. RC2.2 orchestrator (via <code>rc22_adapter.try_orchestrator_first</code>) and DIE analyzer both look at the same command; the winner of <code>deterministic_best_decode</code> is chosen but both ran. Legacy <code>mitre_map</code> regex ALWAYS runs regardless of DIE catalogue results.
M	Is the current IUE capable of expressing "URL → article → text + image → OCR → combined evidence"?	NO. IUE's <code>InputUnderstanding</code> dataclass has no content field, no <code>ocr_intent</code> , no <code>image_ref</code> , no <code>merged_provenance</code> capability. The image adapter with Tesseract exists but is un-imported. Merging OCR-derived and body-derived evidence has no home in the current canonical evidence schema.

§17 · Special multi-stage test — Python -c + Base64 + XOR + docx + exec()

Trace of what happens today (verified against the `PrevMode` saved case, `d2ba2d2e-...`):

INPUT python -c "exec(base64.b64decode(b'...').decode())" afbtDVtsqwFyVTx ■ ▼ Workspace → /api/decode/smart ■ ■■■ _atomic_ioc_kind → not atomic → passthrough ■ ■■■ deterministic_best_decode ■ ■■■ rc22_adapter.try_orchestrator_first (RC2.2 preflight) ■ ■■■ smart_decode + magic_decode ■ ■ peels ONE base64 layer only ■ ■ `^` XOR operator DROPPED from displayed output ■ ■ decoded Python source NOT re-analysed for further decoders ■ ■■■ evidence_extractor.build_verdict_card ■ → operations.mitre_map (regex) → matches "long base64 blob" → T1027 only ■ → risk_score → 65 (Suspicious) ■ ■■■ SAVED case: engine=rc2-orchestrator, mitre=[T1027], reached_shellcode=false Parallel – /api/die/understand: → IUE classify() detects NEITHER `python\_script` (no such type in 21-value enum) → falls to `command\_chain` OR `single\_command` → NO Python-specific plan branch Parallel – /api/die/analyze: → detect_language → "python"? (via services/die/api.py::detect_language) → services/die/python_ast.py runs → missing recognisers: T1059.006, T1027.013, T1140, T1620 → recursive_decode._B64_PATTERNS DOES NOT recognise Python base64.b64decode() → recursive decode NEVER FIRES on this pattern → child artifact "instructions.docx" is REFERENCED IN CODE but NEVER OPENED or ROUTED WHAT'S MISSING today: ■ IUE does not identify Python as a first-class type. ■ Recursive decoder does not recognise Python base64. ■ No artifact router for the referenced "instructions.docx" filename. ■ No child artifact re-enters IUE. ■ DOCX analyzer exists but only via upload route – no way to reach it from a decoded reference. ■ XOR-decrypt pattern (`bytes.fromhex` + XOR loop + exec) not modeled anywhere. ■ T1027.013 (Encrypted/Encoded File) and T1620 (Reflective Code Loading) not in DIE catalogue. ■ Verdict severely under-called (65 Suspicious vs ground-truth Malicious ≥80). WHAT THE TARGET DESIGN would need: ■ IUE with `content: [python, base64, hex, xor\_loop, exec\_pattern]` compound content model. ■ Artifact-expected list: [instructions.docx]. ■ Universal Input Router that ingests the referenced filename as a placeholder artifact and marks it as "unavailable". ■ Recursive discovery loops via IUE for every decoded layer. ■ DIE catalogue additions for T1027.013 / T1620.

Not fixed. Documented only.

\$18 · Partial / dead components (delta since ADR-0012a \$4)

New findings specific to this IUE audit:

Component	Status	Evidence
services/adapters/* — 7 adapters (URL, PDF, DOCX, EML, ZIP, Image, Text)	■ SHADOW — zero production imports	grep "from services.adapters" backend/ → 0 hits
nivxforge/investigation/input_understand.py (291 LoC)	■ STUPID IUE	Called by routers/ops.py:2495 (advisory-only) + routers/auto_investigate.py:739
canonical/iue/plan_builder.py (195 LoC)	■ THIRD IUE	Referenced by canonical/iue/adapters/text_structure.py which bridges into IUE(die)
canonical/iue/composer.py (285 LoC)	■ FOURTH IUE	Similar bridge pattern
services/technique_detector.py NotImplementedError branch	■ SUSPECT DEAD	ADR-0012a \$4.8
services/uil/classifier.py InputKind.STIX / OPENIOC / SIGMA / YARA / PCAP / IMAGE values	■ DEAD ENUM VALUES	Kinds recognised but no analyzer path exists

services/ida/url_intent.py _IOC_PORTAL_HOSTS / _CODE_SNIPPET_HOSTS / _REPO_HOSTS	■ NOT AUDITED THIS PASS	tables likely partial like _VENDORS
_atomic_ioc_kind in v2/investigation/pipeline.py:77	■ SILENT ROUTING DECISION	Runs BEFORE any classifier — no analyst visibility

§19 · IUE bypasses — comprehensive list

Every place where NivXRay makes a routing / classification / decision decision WITHOUT consulting the IUE:

1. **v2/investigation/pipeline.py::_atomic_ioc_kind at line 77** — atomic-IOC short-circuit at the very top of /api/decode/smart. Runs before IUE.
2. **routers/ops.py::decode_smart at line 815-816** — invokes _atomic_ioc_kind and returns immediately on match.
3. **analysis_core.deterministic_best_decode at line 326** — makes decoder-selection decisions (RC2.2 vs smart vs magic) without IUE.
4. **routers/ops.py::decode_smart at line 2495** — IUE is called AFTER decoding, purely to stamp CIO metadata. Its output is not used for routing.
5. **services/files/store::FileStore.put + input_router.route_for** on /api/upload — content-magic routing, IUE bypassed.
6. **routers/behavioral.py::sysmon_ingest and sysmon_evt_x_ingest** — dedicated route bypasses IUE.
7. **services/die/investigation_results.py:322 gate** — makes URL-acquisition decision from IDA classifier, not IUE.
8. **services/die/recursive_decode.merge_evidence** — recursive layer analysis re-enters DIE analyzer, not IUE.
9. **evidence_extractor.build_verdict_card** — calls mitre_map regex + risk_score regardless of IUE opinion.
10. **routers/analyze.py::analyze_async** — enqueues a job without IUE consultation.
11. **AI describe leg on /api/analyze** — LLM output can add MITRE + IOCs; IUE is not consulted.
12. **UIL router /api/uil/classify** — separate classifier endpoint that competes semantically with /api/die/understand.

§20 · Migration boundaries (design-time only)

Ordered by risk. All items require explicit owner authorisation. None are actioned.

Migration	Risk	Prerequisite	Effort
M1 · Add medium.com / systemweakness.com / infosecwriteups.com / posts.specterops.io etc. to _VENDORS catalogue	LOW	none	10-line data edit
M2 · Add realistic browser User-Agent to acquisition requests/httpx calls; harden 403/429 → fall forward to Playwright	LOW	env has Playwright binaries	1 file, ~20 lines

M3 · Install Playwright chromium binaries in preview pod	LOW	none	shell command
M4 · Rewire <code>_next_engine("url_only")</code> to include acquisition step	MED	must preserve existing atomic-IOC fast-path for known-atomic hosts	requires re-thinking the "when to acquire" policy
M5 · Wire <code>services/adapters/*</code> registry as the sole source of adapter routing	HIGH	requires consolidating <code>input_router.route_for</code> + IDA classifier + <code>_atomic_ioc_kind</code> into one call	large architectural change
M6 · Extend <code>InputUnderstanding</code> dataclass with <code>source / content / artifacts_expected / adapter_id / acquisition_intent / ocr_intent</code>	HIGH	breaks the shape of <code>understand()</code> response used by Workspace; requires FE change too	contract-breaking
M7 · Consolidate the 8 classifier modules into ONE authoritative IUE	HIGHEST	requires touching every entry point	multi-session
M8 · Import <code>ImageAdapter</code> and wire it into the acquisition cascade for <code></code> references in acquired body	MED	needs a bytes-fetching sub-cascade; needs provenance fields on canonical evidence	2-3 days

§21 · Critical architectural risks

#	Risk	Consequence	Evidence
R1	Same input, different verdicts across paths	Analysts see contradictory MITRE lists and verdicts depending on which panel they look at. Erodes trust.	Examine case demonstrates this today.
R2	Silent failure of URL acquisition	The Workspace saves an EMPTY case with no diagnostic explaining why.	§12 · six failure modes collapse into one.
R3	IUE plan is analyst-visible AND wrong	On <code>url_only</code> the IUE tells the analyst "IOC Enrichment + Report Generator" — technically what happens, but omits the acquisition step the analyst assumes will run.	<code>_engines_selected("url_only") = ["IOC Enrichment", "Report Generator"]</code> .
R4	Two IUEs disagree at times	<code>services/die/input_understanding</code> and <code>nivxforge/investigation/input_understanding.py</code> have independent classifiers with different type enums.	The <code>die</code> one is 291 LoC with its own logic.

R5	Recursive discovery does not re-enter IUE	Child artifacts (peeled base64, decoded XOR, extracted DOCX) never get authoritative classification or routing.	<code>services/die/recursive_decode.py</code> re-enters DIE, not IUE.
R6	OCR-derived evidence has no provenance schema	If OCR were wired in tomorrow, OCR text would be indistinguishable from authoritative body text. Evidence integrity risk.	Canonical evidence has no <code>extraction_method</code> or <code>source="image_ocr"</code> value.
R7	Legacy MITRE regex still authoritative for the primary route	Saved cases carry the regex mapper's opinion, not the DIE catalogue's.	PrevMode case has <code>mitre=[T1027 (regex)]</code> ; Examine has <code>mitre=[]</code> .
R8	_VENDORS catalogue is a policy-critical hard-coded table	Adding one new SOC blog requires a code deploy. No admin-editable list.	<code>services/ida/url_intent.py:53-96</code>
R9	The <code>services/adapters/*</code> shadow registry could rot	7 well-structured adapters exist but no CI checks them. They may already be silently broken.	Zero production imports; no adapter-suite regression test.
R10	Playwright fallback silently returns "	The last-resort acquisition engine can appear to succeed while returning nothing.	<code>services/ida/acquisition.py:498</code> "silently returns " if Playwright is unavailable".

§22 · Recommended next steps — NO IMPLEMENTATION

Sequence (each item requires explicit owner authorisation before starting):

1. **■ Ratify this audit** — publish ADR-0013 pinning the current architecture, the 8-classifier landscape, and the IUE responsibility gap.
2. **■ Decide the target: single-IUE convergence vs. keep-parallel-but-document.** Everything else depends on this choice.
3. **■ Freeze the IUE dataclass shape** before extending it. The current shape is consumed by both Workspace UI and multiple internal callers.
4. **■ Design (not implement) the extended IUE contract** — `source / content / artifacts_expected / adapter_id / acquisition_intent / ocr_intent` — and circulate for owner review.
5. **■ Design (not implement) the Universal Input Router** — a thin dispatcher that consumes the extended IUE decision and maps to `services/adapters/*` registry entries. Preserve every existing analyzer.
6. **■ Enumerate the migration cost** for switching Workspace `/api/decode/smart` from RC2 to IUE-driven canonical entry.
7. **■ Owner check-point** — decide whether to attempt Migration M1-M3 (low-risk vendor catalogue + User-Agent + Playwright binary) as a "prove the acquisition lane" step, WITHOUT touching IUE.
8. **Stop.** Do not authorise M4-M8 until M1-M3 confirms the acquisition layer produces real evidence.

Locks that remain intact:

- Sysmon Event 22 · Event 11 — LOCKED

- Sandbox parser boundary — LOCKED
- Real Investigation Proof Phase B — LOCKED
- Verdict Engine v3 promotion — LOCKED
- New MITRE mappers / verdict logic / behavioral rules — LOCKED
- Workspace reroute — LOCKED
- OCR implementation — LOCKED (design only)
- Playwright install — LOCKED (proposed but not authorised)
- User-Agent change — LOCKED (proposed but not authorised)
- Vendor catalogue extension — LOCKED (proposed but not authorised)

§23 · Rules confirmation

- ■ No file modified during this audit. Only two additive deliverables produced (this markdown + the PDF export via the earlier `/api/audit/*` route).
- ■ Every claim carries a file / function / line citation. Anything unverifiable is tagged `NOT VERIFIED FROM CODE`.
- ■ No new component created. No new intelligence layer proposed.
- ■ Task 2 (Real EVTX Fixture) remains COMPLETE.
- ■ Task 3 (auto-scroll) and Task 4 (source-agnostic audit) remain LOCKED.
- ■ UI-DEF-02 authoritative surface untouched. No MITRE / verdict / adapter / IKG / Workspace change.

End of IUE 360° READ-ONLY audit. STOP.